ASL Recognition System

DA5402:Machine Learning Operations Lab

American Sign Language Classification
with End-to-End MLOps Pipeline

Name	Neer Patel
Roll no	DA25M021

Dataset	Kaggle — `mayank0bhardwaj/alphabets`
Model Family	CNN (best), MLP, KNN, Random Forest
Tech Stack	FastAPI, MLflow, DVC, Airflow, Prometheus, Grafana
Container	Docker Compose (API + Monitoring + Airflow stacks)

0 Contents

1 Introduction & Problem Statement **3**
1.1 Problem Statement . . . 3
1.2 Business Objectives . . . 3
1.3 Success Metrics . . . 4
1.4 Contributions . . . 4

2 Architecture **5**
2.1 High-Level Architecture . . . 5
2.2 Docker Compose Services . . . 6

3 Data Engineering Pipeline **7**
3.1 Airflow DAG: `asl_data_ingestion` . . . 7
3.1.1 Task Descriptions . . . 8
3.2 Airflow Configuration . . . 9

4 DVC Pipeline & Source Control **10**
4.1 DVC DAG . . . 10
4.2 DVC Stages Summary . . . 11
4.3 Versioning Strategy . . . 11
4.4 MLproject Entry Points . . . 11

5 Feature Engineering **12**
5.1 Feature Types . . . 12
5.2 Landmark Schema . . . 12
5.3 Augmentation Parameters . . . 12

6 Model Development & Experiment Tracking **13**
6.1 Model Architectures . . . 13
6.1.1 Tiny-CNN (Best Performer) . . . 13
6.1.2 Landmark-MLP . . . 13
6.1.3 Traditional ML (KNN) . . . 14
6.1.4 Traditional ML (Random Forest) . . . 14
6.2 Experimental Results . . . 14
6.3 MLflow Experiment Tracking . . . 14
6.4 Per-Class F1 (Tiny-CNN) . . . 16
6.5 MLflow Tracked Artefacts . . . 16

7 FastAPI Backend — Endpoint Specification (LLD) **17**
7.1 Public Endpoints . . . 17
7.2 Admin Endpoints (HTTP Basic Auth) . . . 17
7.3 Model Serving API . . . 17
7.4 User Interface . . . 18
7.5 Admin Dashboard . . . 18

8 Monitoring Architecture **19**
8.1 Prometheus Instrumentation . . . 19
8.2 Monitoring Architecture Diagram . . . 19

8.3 Prometheus Dashboard 20
8.4 Grafana Dashboards 21

9 Alerting Rules **23**
9.1 Alert Rule Table 23
9.2 Alertmanager Configuration 23

10 References **25**

1 Introduction & Problem Statement

Communication barriers for the deaf and hard-of-hearing community remain a significant challenge, particularly in scenarios where sign language interpreters are unavailable. Automatic recognition of American Sign Language (ASL) gestures using computer vision and machine learning offers a scalable solution for improving accessibility in education, healthcare, and human-computer interaction systems.

In this work, we present an end-to-end ASL Recognition System that classifies static hand gesture images into one of 27 categories (letters A–Z and a `NIL` class for non-gestures). Unlike isolated model-focused studies, this project is designed as a complete production-grade system, integrating data ingestion, training, deployment, and monitoring into a unified MLOps pipeline.

The system leverages a combination of deep learning and traditional machine learning approaches, including a lightweight convolutional neural network (Tiny-CNN), a landmark-based multi-layer perceptron (MLP), and classical models such as K-Nearest Neighbours and Random Forests. Among these, the Tiny-CNN model demonstrates the best performance and is selected for deployment.

Beyond model development, a key focus of this project is reproducibility and operational reliability. The pipeline incorporates:

- Automated data ingestion and validation using Airflow
- Data and model versioning with DVC
- Experiment tracking and model registry via MLflow
- Real-time inference through a FastAPI-based service
- Monitoring, alerting, and drift detection using Prometheus, Grafana, and Alertmanager

This integrated design ensures that the system is not only accurate but also maintainable, observable, and scalable in real-world deployment scenarios.

1.1 Problem Statement

The primary objective of this project is to design and implement a robust ASL classification system that:

- Accurately classifies hand gestures from images into predefined ASL categories
- Supports low-latency inference suitable for interactive applications
- Maintains performance under changing data distributions through monitoring and retraining
- Ensures full reproducibility of experiments and model artifacts

1.2 Business Objectives

- Provide a low-latency (<200 ms) ASL classification API usable by accessibility tools and educational applications.

- Enable retraining when data distribution drifts beyond a PSI threshold of 0.3.
- Maintain system uptime with automated alerting and monitoring.
- Keep all artefacts (data, models, configs) fully versioned and reproducible.

1.3 Success Metrics

Metric	**Target**	**Achieved (CNN)**
Macro F1 (test set)	≥ 0.30	0.3157
Inference latency p99	$< 2\,\mathrm{s}$	$\sim$150 ms
Error rate	$< 5\%$	Monitored via Prometheus
Drift PSI alert	> 0.3	Configured in alert rules

1.4 Contributions

The key contributions of this work are:

- Design of a complete end-to-end MLOps pipeline for ASL recognition
- Comparative evaluation of deep learning and traditional ML approaches
- Deployment of a real-time inference system with monitoring and alerting
- Integration of drift detection and automated retraining triggers

2 Architecture

2.1 High-Level Architecture

Figure 1,2 and 3 shows the full system decomposed into five planes: Data, Training, Serving, Monitoring, and Orchestration.

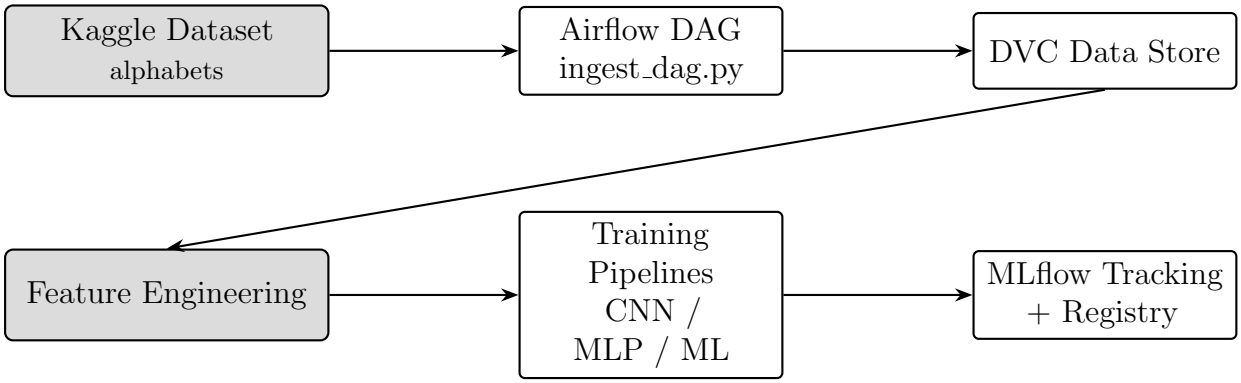


Figure 1: Data Ingestion and Training Pipeline

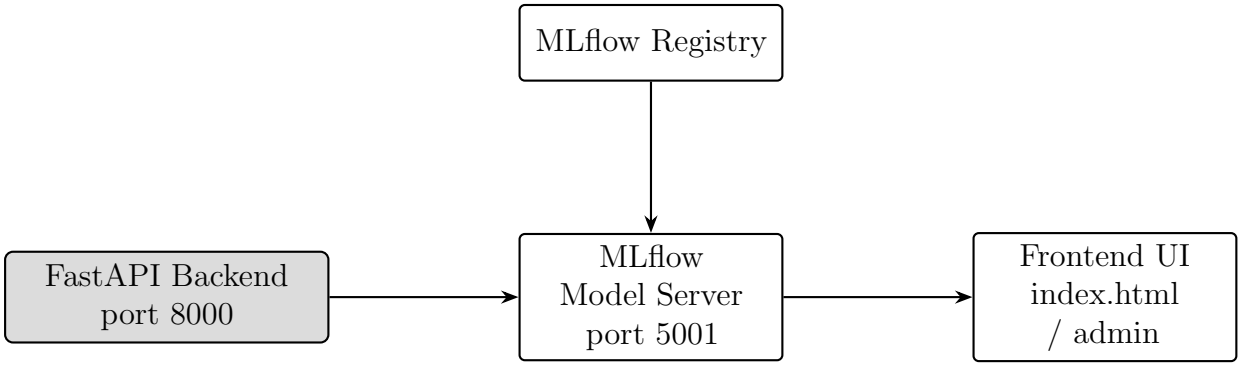


Figure 2: Model Serving Architecture

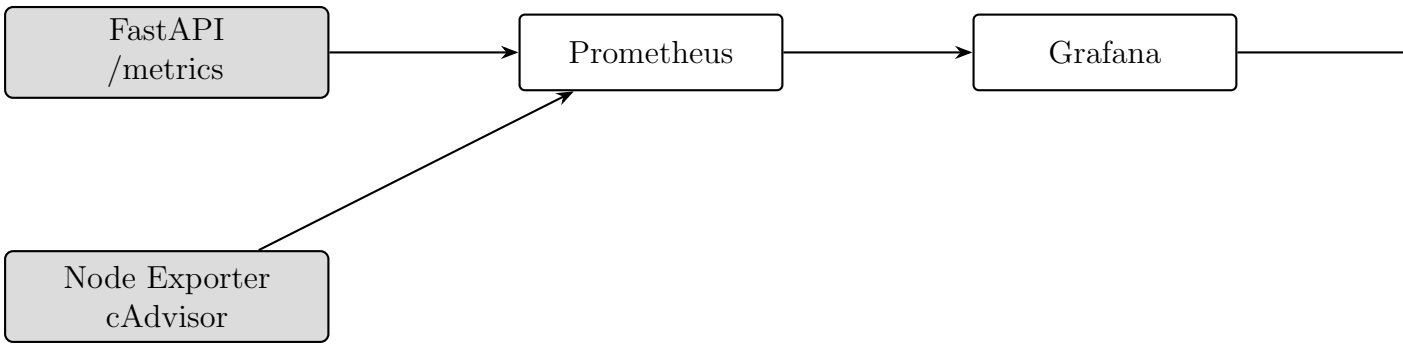


Figure 3: Monitoring and Alerting Stack

2.2 Docker Compose Services

Two Docker Compose stacks are used:

Service	Image	Port	Purpose
`api`	custom (Dockerfile.api)	8000	FastAPI inference + admin
`prometheus`	`prom/prometheus:2.51`	9090	Metrics scraping + rules
`alertmanager`	`prom/alertmanager:0.27`	9093	SMTP email routing
`grafana`	`grafana/grafana:10.4.2`	3000	Dashboards (NRT)
`node_exporter`	`prom/node-exporter:1.7`	—	Host OS metrics
`cadvisor`	`gcr.io/cadvisor:0.49`	—	Container metrics
`airflow-webserver`	`apache/airflow:2.x`	8080	DAG management UI
`airflow-scheduler`	same	—	Scheduler
`postgres`	`postgres:13`	5432	Airflow metadata DB

3 Data Engineering Pipeline

3.1 Airflow DAG: `asl_data_ingestion`

The data ingestion pipeline is implemented as a 7-task Airflow DAG. Figure 4 shows the actual DAG as rendered in the Airflow web UI, and Figure 6 shows the linear task dependency diagram.

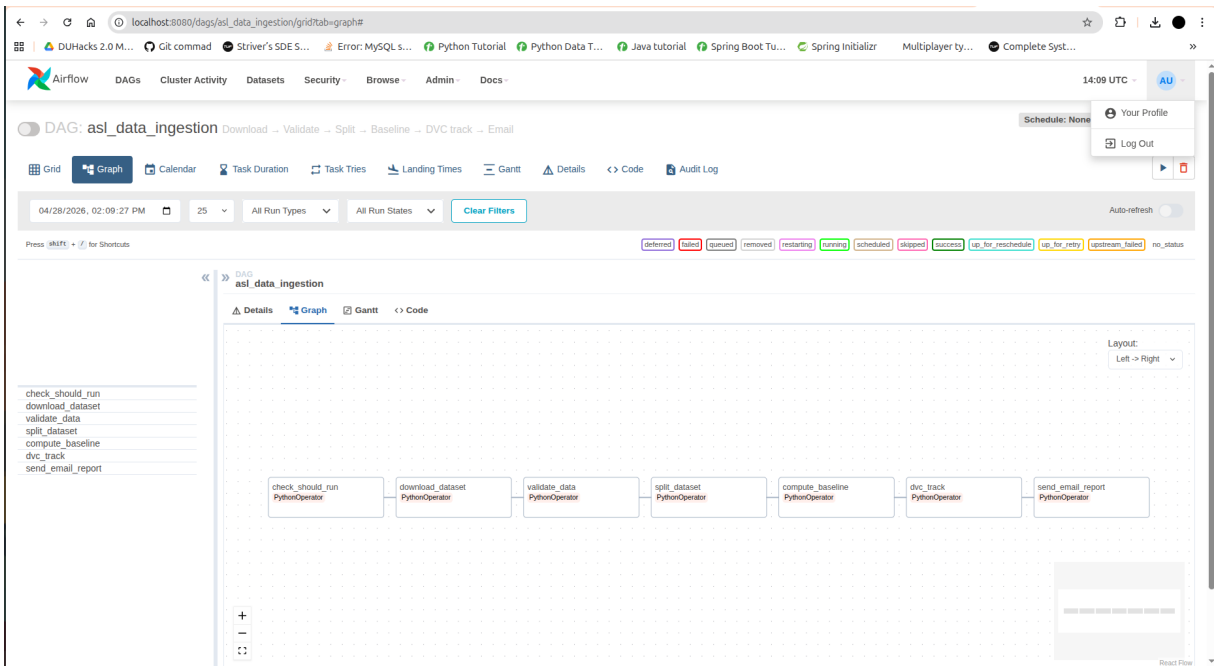


Figure 4: Airflow DAG — `asl_data_ingestion` as rendered in the Airflow UI

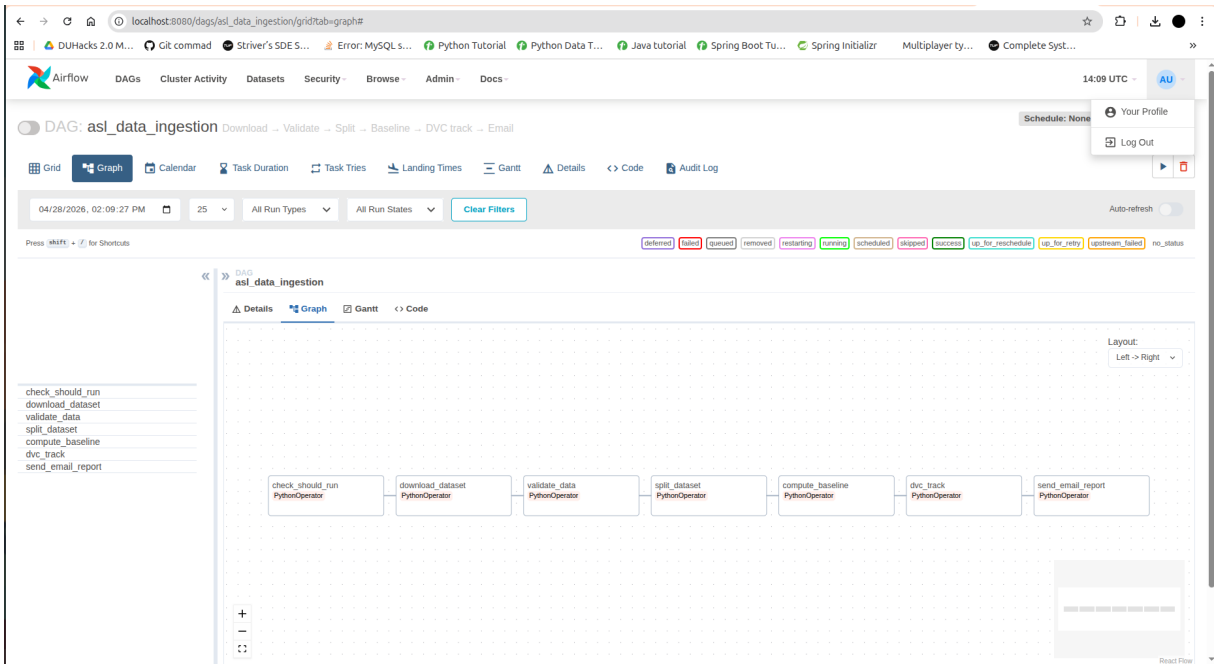


Figure 5: Airflow DAG — `asl_data_ingestion` as rendered in the Airflow

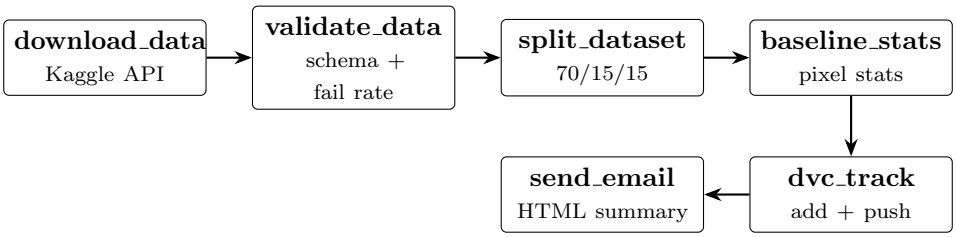


Figure 6: Airflow DAG — `asl_data_ingestion` task graph

3.1.1 Task Descriptions

Task	Description
`check_should_run`	Guard task. Computes MD5 hash of `params.yaml`. Skips all downstream tasks if data already exists and hash is unchanged (unless `force_rerun: true`).
`download_dataset`	Downloads the ASL alphabet dataset from Kaggle (`mayank0bhardwaj/alphabets`) using the Kaggle API. Returns image count, class count, and per-class counts via XCom.
`validate_data`	Checks image integrity, verifies expected class folders (A–Z + NIL), counts failed images. Raises `ValueError` if `fail_rate > max_fail_rate` from `params.yaml`.
`split_dataset`	Stratified train/val/test split (default 70/15/15). Results pushed to XCom.
`compute_baseline`	Calculates global pixel mean and variance over the training split. Saved to `data/baseline_stats.json` for downstream drift detection.

Task	Description
`dvc_track`	Runs `dvc add data/raw data/processed` and `dvc push`. Writes a marker file (`.dvc_last_run.json`) with the params hash and timestamp.
`send_email_report`	Sends an HTML email summarising download stats, validation results, split counts, label distribution, baseline stats, and DVC tracking metadata.

3.2 Airflow Configuration

```
default_args:
  owner: "asl-project"
  email_on_failure: true
  retries: 1
  retry_delay: 2 minutes
schedule_interval: None   # manually triggered or via params hash change
```

Listing 1: Airflow DAG default_args

4 DVC Pipeline & Source Control

4.1 DVC DAG

DVC manages the ML training pipeline via a `dvc.yaml` file. The stages form a DAG as shown in Figure ??.

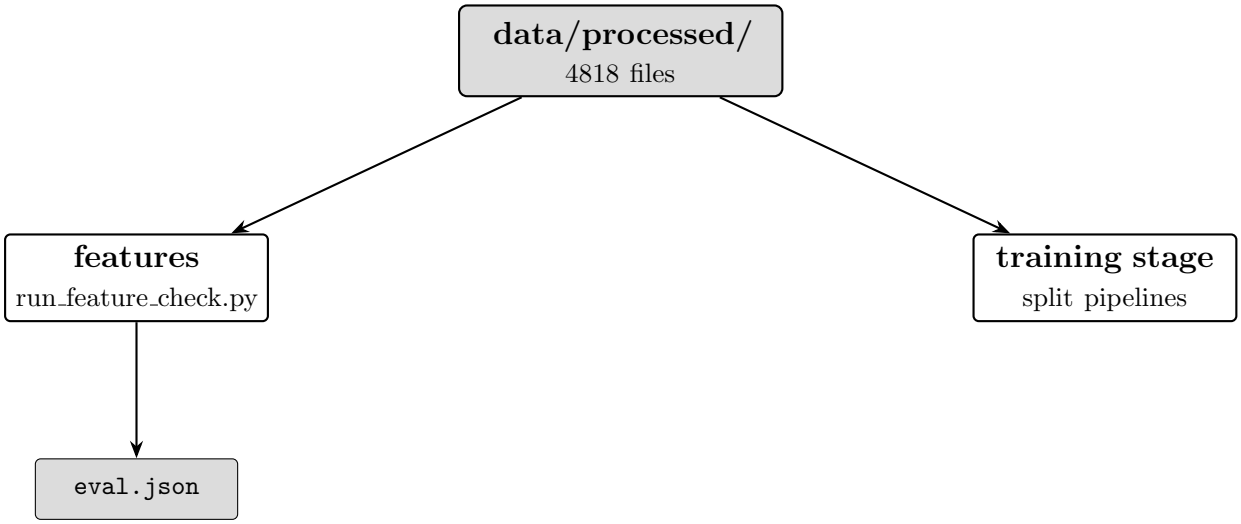


Figure 7: Data Processing and Feature Validation

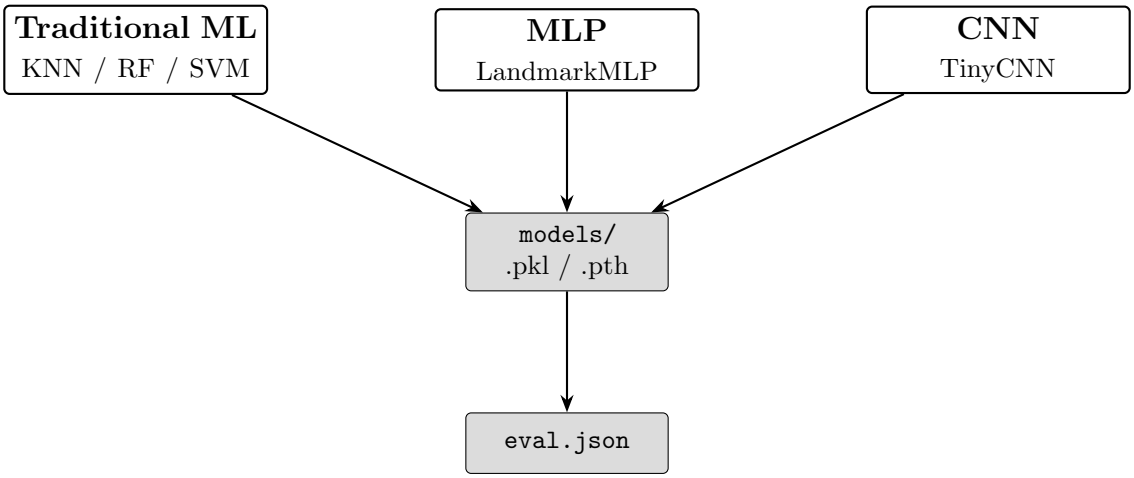


Figure 8: Model Training Pipelines and Outputs

4.2 DVC Stages Summary

Stage	Command	Key Parameters
`features`	`python src/features/run_feature_check.py`	image_size=224, normalize_mean=[0.5,0.5,0.5], augmentations
`train_traditional`	`python src/train.py --model traditional_ml`	algorithm=knn, n_neighbors=7, weights=distance, metric=euclidean
`train_mlp`	`python src/train.py --model mlp`	hidden=[256,128], dropout=0.4, batch_norm=True, epochs=20, lr=0.001
`train_cnn`	`python src/train.py --model cnn`	channels=[16,32,64,96], fc_dim=128, dropout=0.3, epochs=20, lr=0.001

4.3 Versioning Strategy

- **Code:** Git (all `src/`, `dags/`, `app/`, `monitoring/` tracked).
- **Data:** DVC — `data/raw` and `data/processed` tracked as `.dvc` pointer files; actual bytes pushed to a local DVC store.
- **Models:** DVC tracks `models/cnn/cnn_best.pth`, `models/mlp/mlp_best.pth`, `models/traditional` Additionally registered in MLflow Model Registry.
- **Experiments:** Every training run is logged to MLflow (parameters, metrics, confusion matrix artefact). Reproducibility: Git commit hash + MLflow `run_id`.

4.4 MLproject Entry Points

```
entry_points:
  train:       python src/train.py --model {model} --lr {lr} ...
  download:    python src/data/download.py --dest_dir {dest_dir}
  validate:    python src/data/validate.py --dataset_path {path}
  split:       python src/data/split.py --dataset_path {path}
  full_pipeline: dvc repro   # MLflow launches DVC
```

Listing 2: MLproject entry points

5 Feature Engineering

5.1 Feature Types

Type	File	Shape	Latency	Notes
Hand landmarks	`landmarks.py`	(63,)	∼15 ms	MediaPipe Hands; wrist-origin normalised; None if no hand detected
Raw pixels	`pixels.py`	(3,224,224)	∼5 ms	torchvision transforms; ImageNet normalisation
Pixels + bg removal	`bg_remove.py`	(3,224,224)	∼500 ms	rembg (U2-Net) or GrabCut fallback

5.2 Landmark Schema

21 3D keypoints from MediaPipe Hands, flattened to a 63-dimensional vector $[x_0, y_0, z_0, \ldots, x_{20}, y_{20}, z_{20}]$. The wrist (index 0) is used as the origin after subtraction, and the result is scaled to $[-1, 1]$.

5.3 Augmentation Parameters

```yaml
preprocessing:
  image_size: 224
  normalize_mean: [0.5, 0.5, 0.5]
  normalize_std:  [0.5, 0.5, 0.5]
  augmentations:
    random_rotation_degrees: 15
    horizontal_flip: true
    color_jitter_brightness: 0.3
```

Listing 3: Augmentation parameters (params.yaml)

6 Model Development & Experiment Tracking

6.1 Model Architectures

6.1.1 Tiny-CNN (Best Performer)

A lightweight convolutional neural network operating on pixel features. Channels: [16, 32, 64, 96]; FC dimension: 128; Dropout: 0.3; Input: $(3, 224, 224)$. Trained with Adam, cosine LR scheduler, label smoothing 0.1.

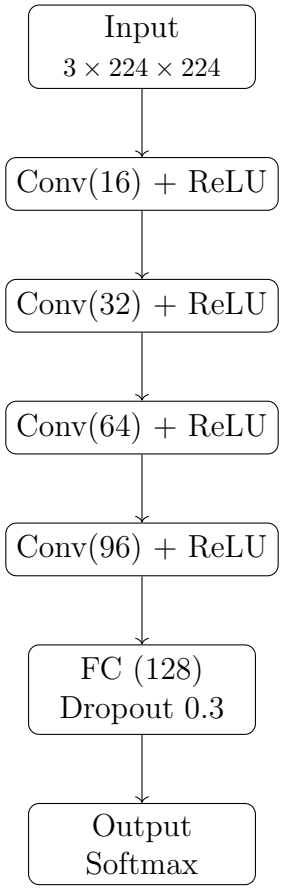


Figure 9: Tiny-CNN Architecture

6.1.2 Landmark-MLP

A fully-connected network on the 63-dimensional landmark vector. Hidden layers: [256, 128]; Batch normalisation; Dropout: 0.4.

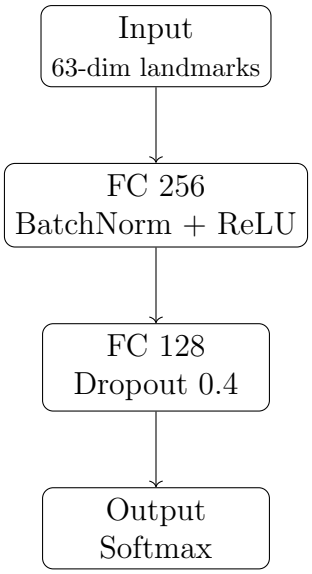


Figure 10: Landmark-MLP Architecture

6.1.3 Traditional ML (KNN)

KNN with $k = 7$, distance weighting, Euclidean metric, operating on landmark features.

6.1.4 Traditional ML (Random Forest)

200 estimators, no max depth, min_samples_split=2.

Model	Key Parameters	Features
KNN	$k = 27$, distance-weighted	Landmarks (63D)
Random Forest	200 trees, no depth limit	Landmarks (63D)

Table 2: Traditional Machine Learning Models

6.2 Experimental Results

Model	**Accuracy**	**Macro F1**	**Notes**
Tiny-CNN	**35.46%**	**0.3157**	Best overall; pixel features
KNN	4.43%	0.0171	Landmark features; overfits classes
Random Forest	4.85%	0.0228	Landmark features
Landmark-MLP	4.43%	0.0148	Landmark features

Note: The large accuracy gap between CNN (pixel) and landmark-based models indicates that the MediaPipe landmark extractor struggles on this dataset — likely due to image quality or background complexity preventing reliable hand detection. The CNN operating directly on pixels bypasses this bottleneck.

6.3 MLflow Experiment Tracking

The following screenshots show the MLflow tracking server as used during experimentation.

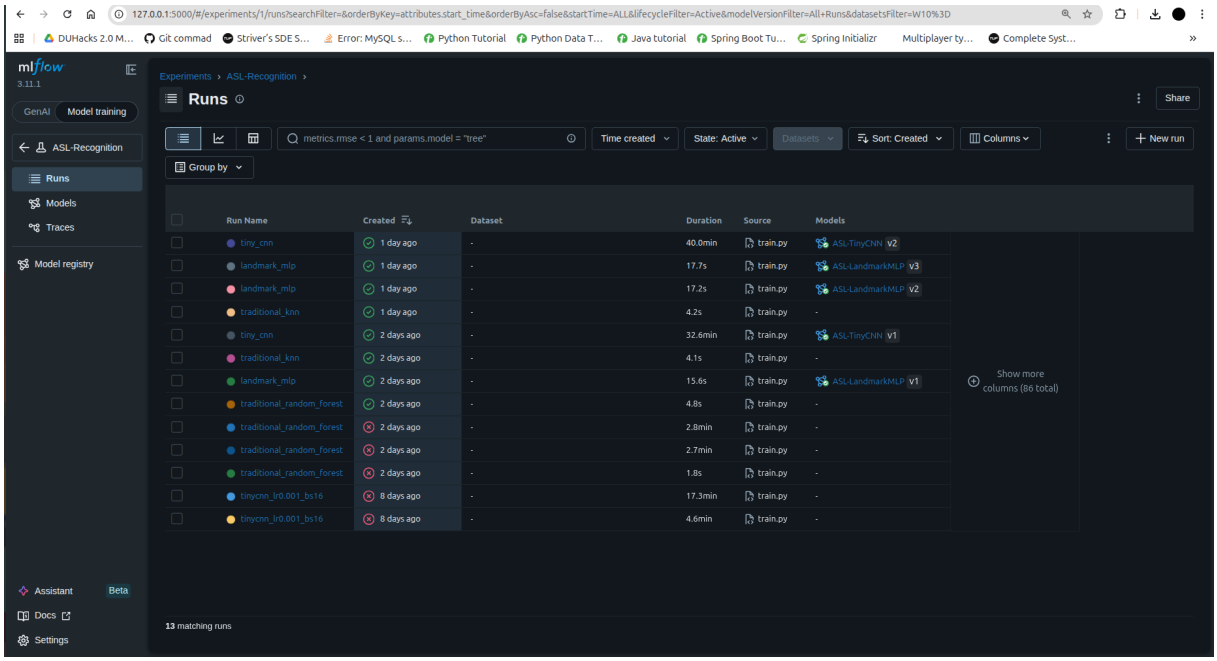


Figure 11: MLflow Runs — All experiment runs listed in the tracking UI

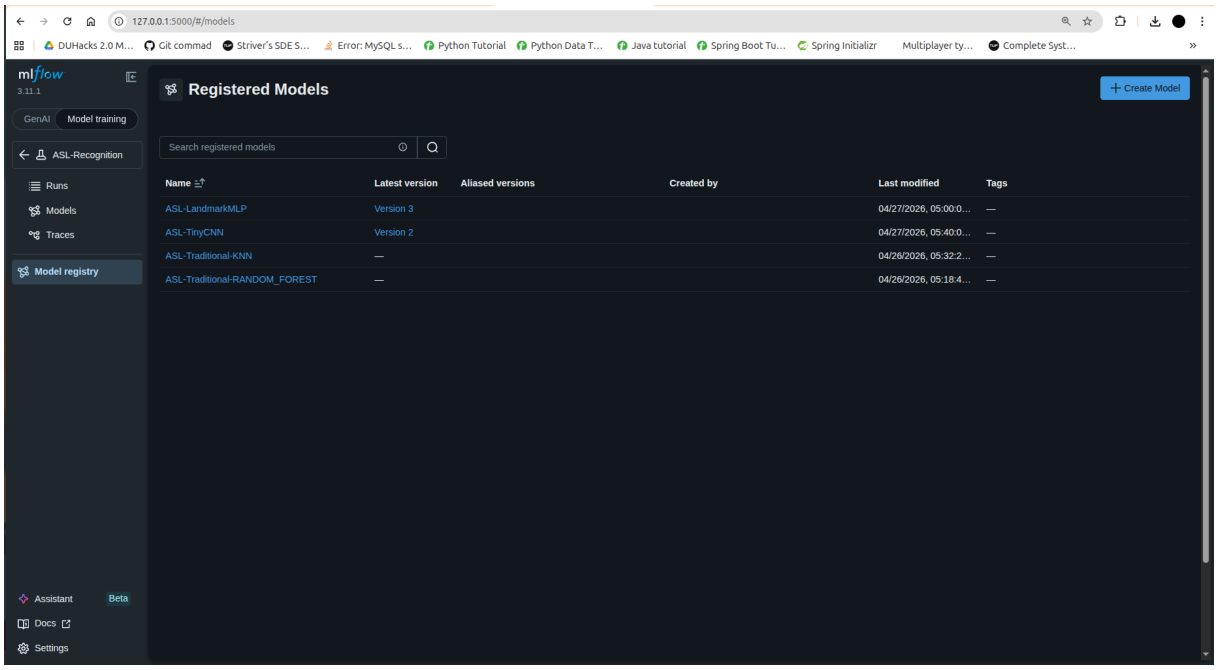
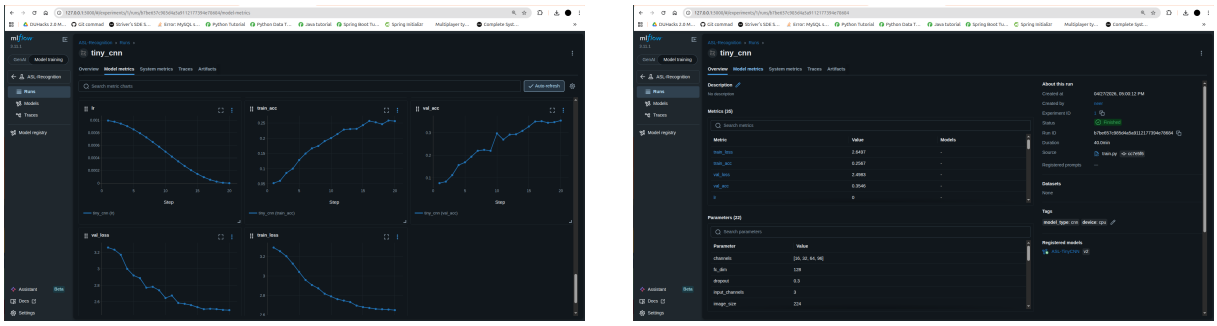


Figure 12: MLflow Model Registry — Versioned model artefacts



(a) CNN Training Metrics **(b)** CNN Run Details

Figure 13: Tiny-CNN Experiment — MLflow metrics and run detail view

6.4 Per-Class F1 (Tiny-CNN)

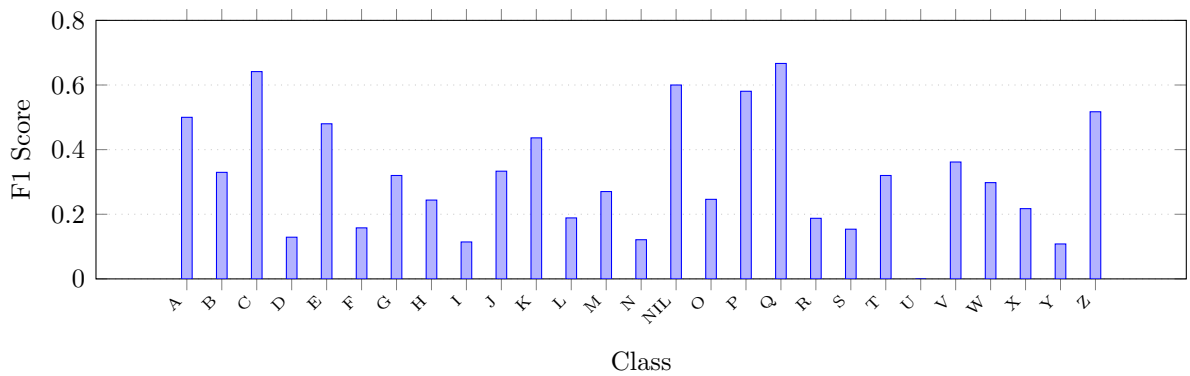


Figure 14: Per-class F1 score — Tiny-CNN on test set. Classes Q (0.667), C (0.641), NIL (0.600) perform best; U (0.000), N (0.121), D (0.129) perform worst.

6.5 MLflow Tracked Artefacts

- **Parameters:** model type, lr, epochs, batch_size, dropout, hidden dims, channels, augmentation config.
- **Metrics:** accuracy, macro F1, per-class F1 (all 27 classes).
- **Artefacts:** confusion matrix PNG per model (`confusion_matrix/{model}_confusion_matrix.png`).
- **Model Registry:** models registered as versioned MLflow models with `MLmodel`, `conda.yaml`, `requirements.txt`.
- **Autolog + manual:** MLflow autolog captures framework-level metrics; manual `mlflow.log_metric` / `log_artifact` calls add domain-specific data.

7 FastAPI Backend — Endpoint Specification (LLD)

7.1 Public Endpoints

Method	Path	Description	Response
GET	`/`	Serve index.html UI	HTML
GET	`/health`	Health check	{`status: "ok"`}
GET	`/ready`	Readiness check	{`ready: true`}
POST	`/predict`	Upload image, return predicted class + confidence	{`label, confidence, latency_ms`}
GET	`/metrics`	Prometheus metrics	plain text

7.2 Admin Endpoints (HTTP Basic Auth)

Method	Path	Description	Response
GET	`/admin`	Serve admin dashboard HTML	HTML
GET	`/admin/mlflow/experiments`	List all MLflow experiments and runs	JSON
GET	`/admin/mlflow/models`	List registered models + versions	JSON
GET	`/admin/mlflow/metrics`	Aggregate metrics from local JSON files	JSON

7.3 Model Serving API

The model is additionally exposed via the MLflow model server. Figure 15 shows the auto-generated Swagger/OpenAPI documentation for the serving endpoint.

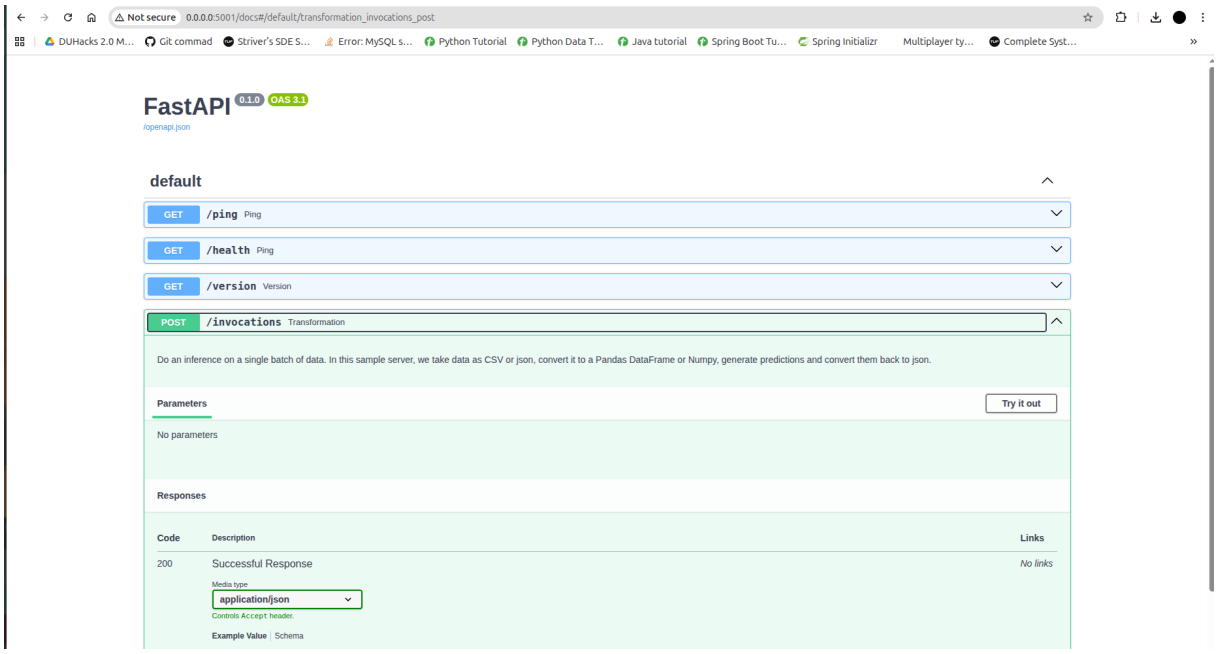
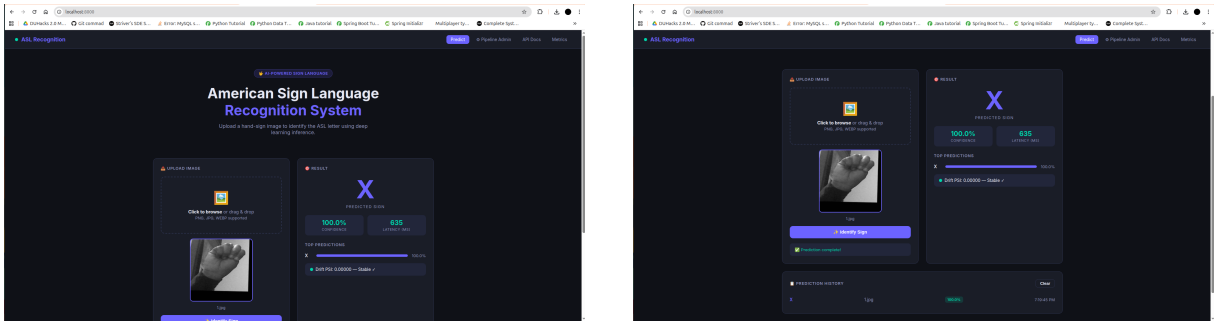


Figure 15: Model Serving API — Swagger/OpenAPI UI at the MLflow model server (port 5001)

7.4 User Interface

The web UI allows users to upload a hand-gesture image and receive a predicted ASL letter with a confidence score. Figure 16 shows the main interface and the prediction output view.

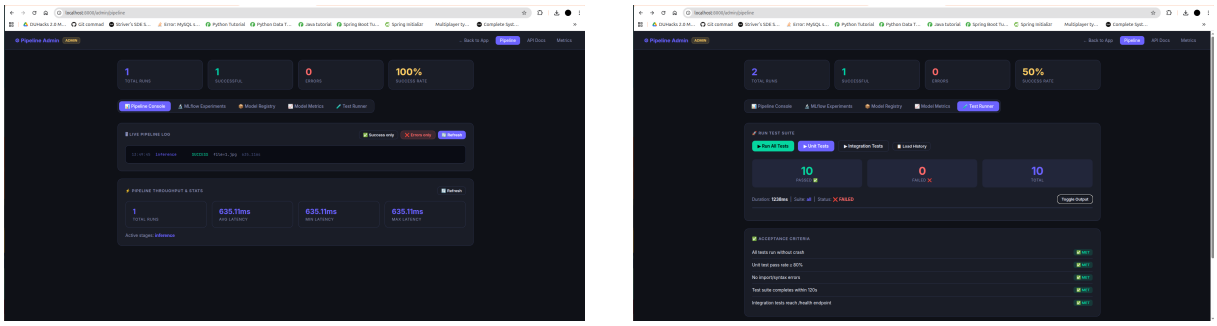


(a) Main Upload UI **(b)** Prediction Output

Figure 16: User Interface — Main screen and prediction result view

7.5 Admin Dashboard

The admin dashboard provides authenticated access to experiment runs, model registry information, and the ability to trigger test suites directly from the browser.



(a) Admin Panel **(b)** Test Execution View

Figure 17: Admin Dashboard — Panel overview and test execution screen

8 Monitoring Architecture

8.1 Prometheus Instrumentation

The FastAPI application exposes the following custom metrics at `/metrics`:

Metric Name	Type	Description
`prediction_requests_total`	Counter	Total inference requests
`prediction_errors_total`	Counter	Total inference errors
`prediction_latency_seconds`	Histogram	Inference latency (p50/p99) Buckets: [0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0]
`model_drift_score`	Gauge	Current PSI drift score
`pipeline_runs_total`	Counter	Pipeline events by stage/status
`test_runs_total`	Counter	Test suite executions by status

In addition, `prometheus_fastapi_instrumentator` auto-instruments all HTTP endpoints (request count, latency histograms, status codes) and exposes them at `/metrics`. Node Exporter and cAdvisor provide host-level CPU, memory, disk, and container metrics.

8.2 Monitoring Architecture Diagram

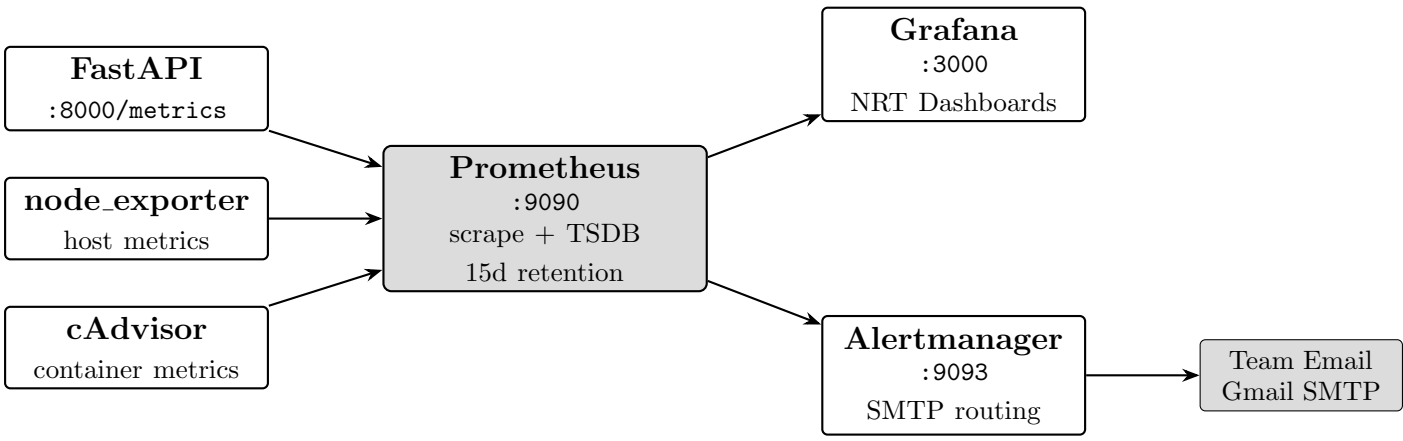


Figure 18: Monitoring architecture — Prometheus scrape topology

8.3 Prometheus Dashboard

Figures 19 through 21 show the live Prometheus web UI, the metrics graph view, and the configured alert rules.

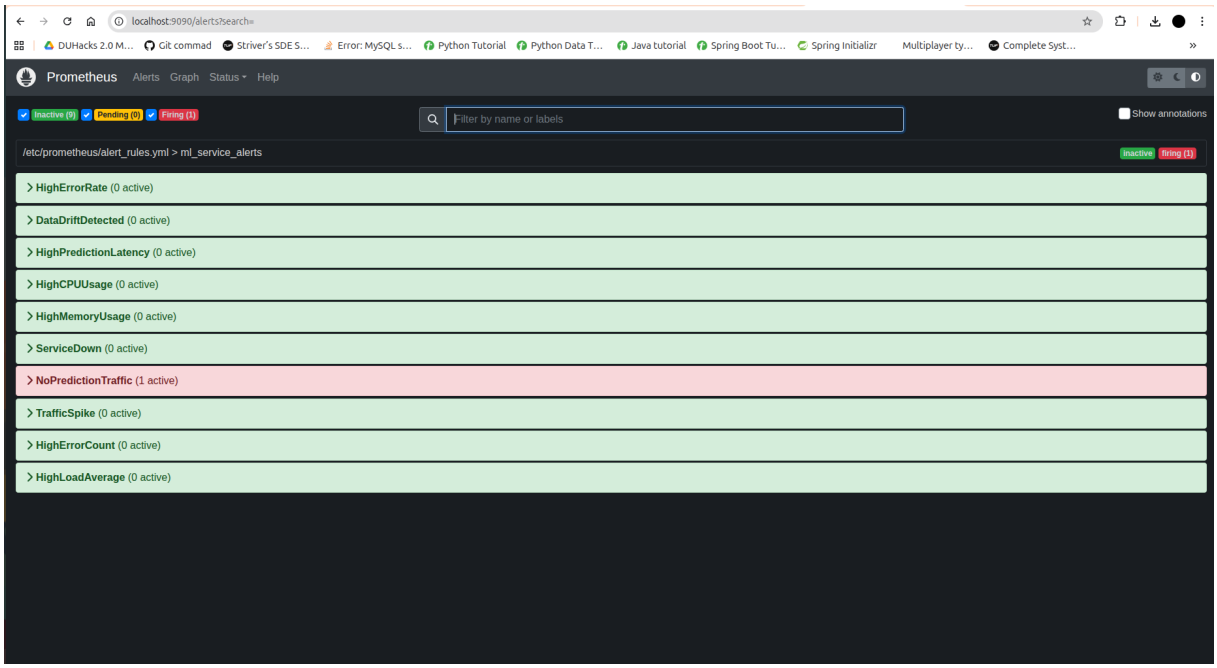


Figure 19: Prometheus Dashboard — Targets and scrape status

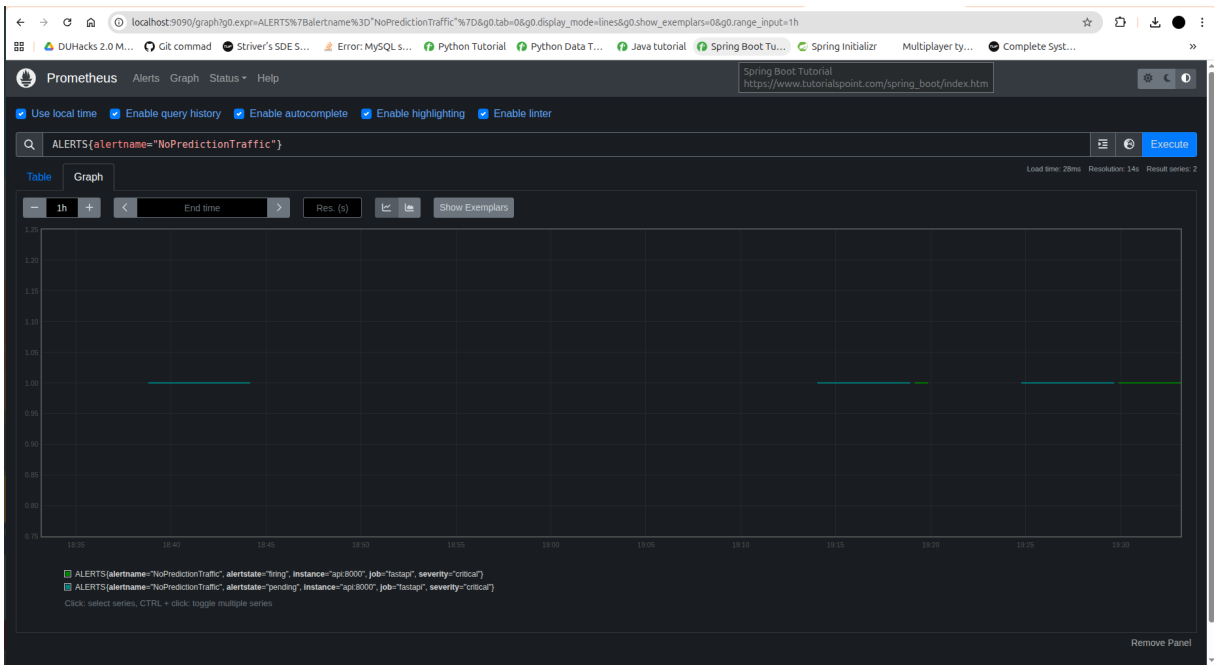


Figure 20: Prometheus — Metrics graph / expression browser

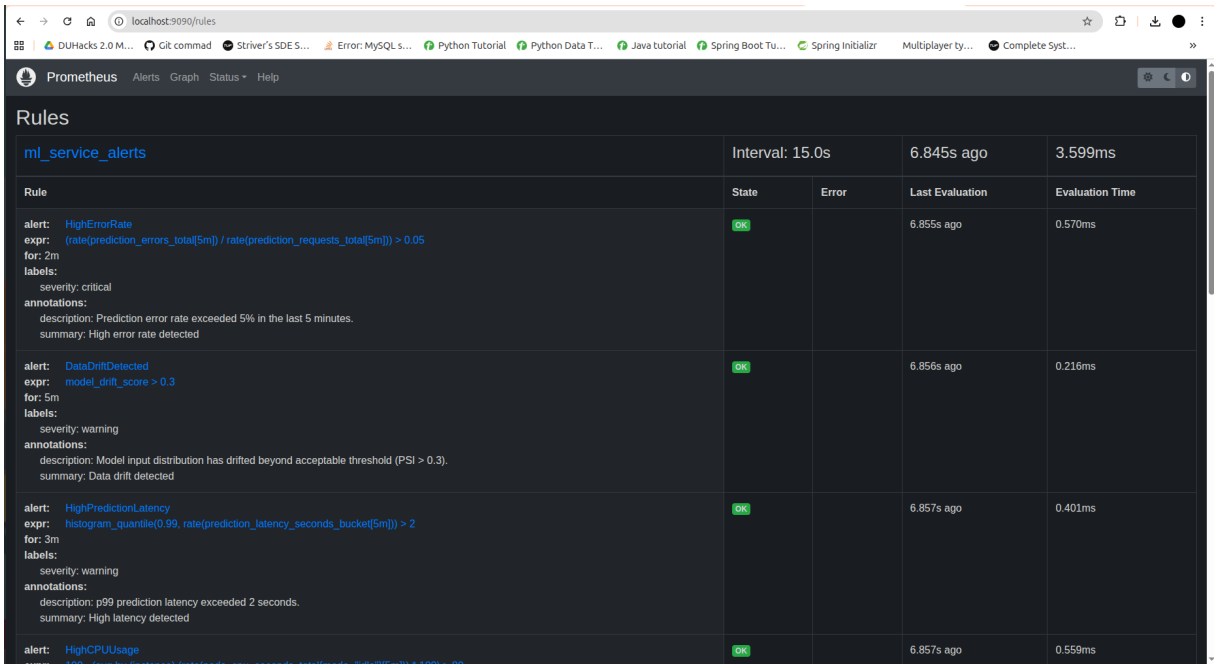


Figure 21: Prometheus — Loaded alert rules

8.4 Grafana Dashboards

Three Grafana dashboard panels provide near-real-time visibility into system health, model performance, and infrastructure metrics (Figure ??).

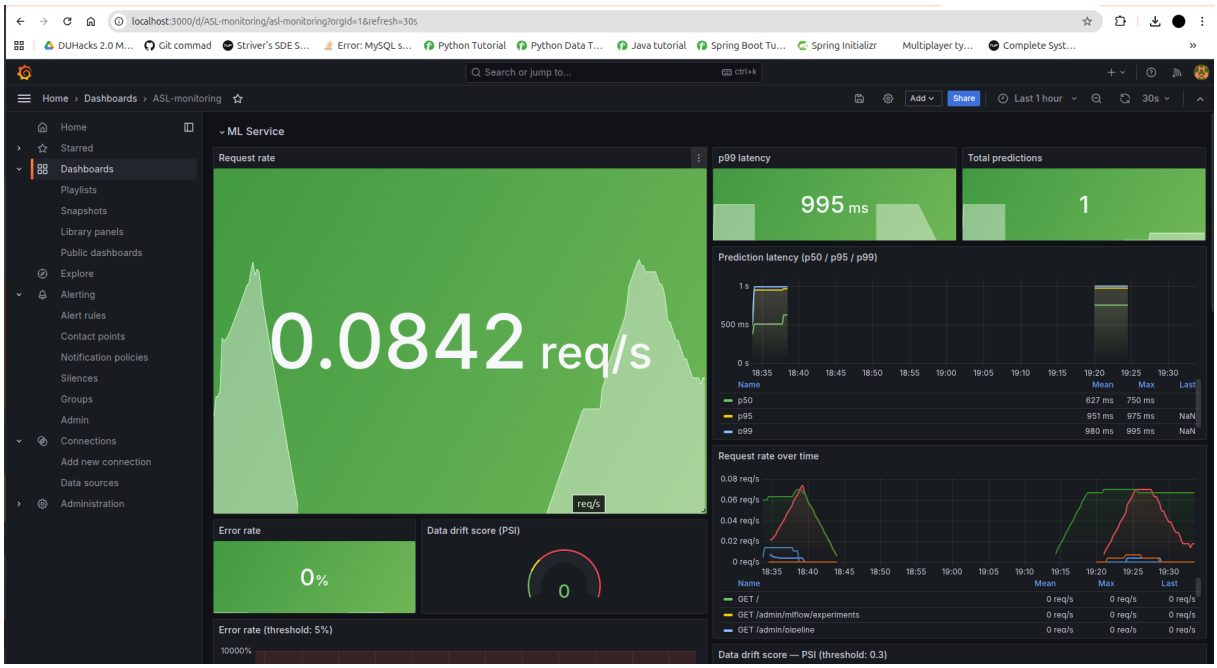


Figure 22: Grafana Dashboard — System Overview

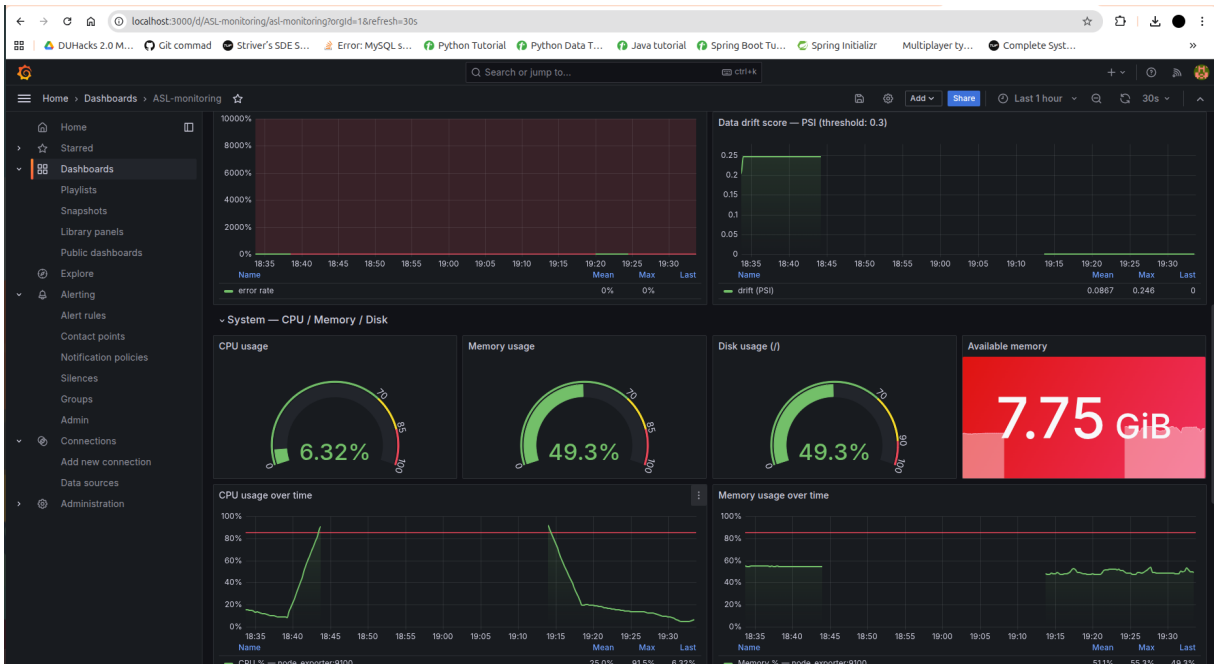


Figure 23: Grafana Dashboard — Model Performance Metrics

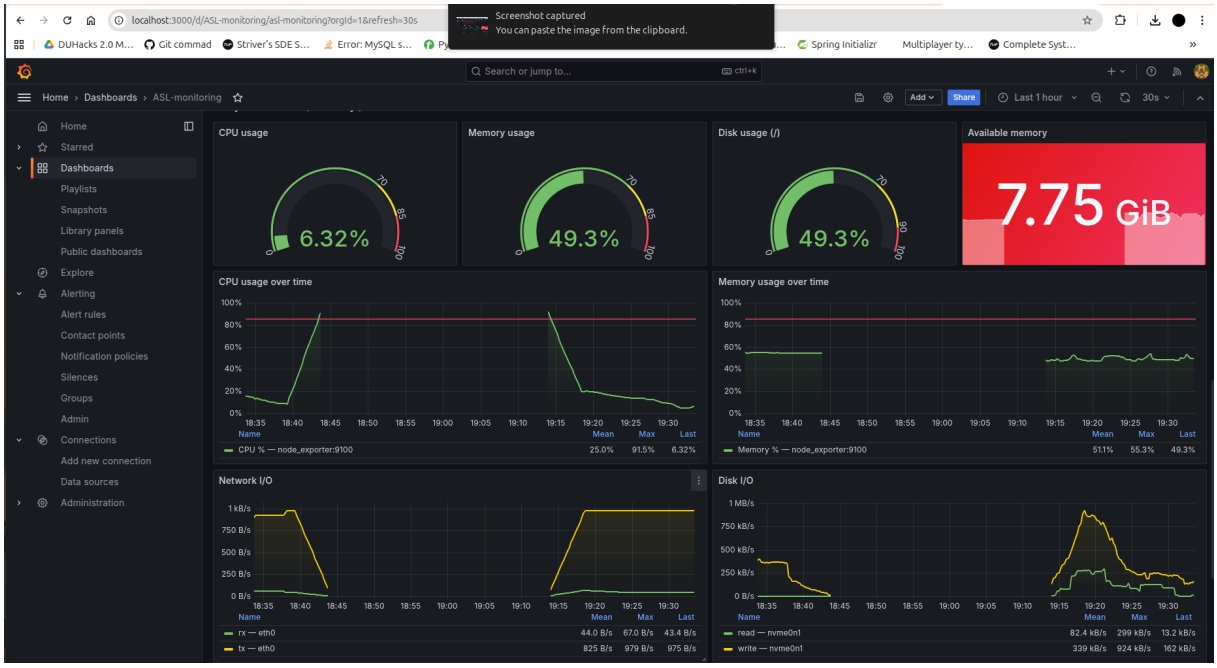


Figure 24: Grafana Dashboard — Infrastructure Monitoring

9 Alerting Rules

All rules are in `monitoring/alert_rules.yml` and loaded by Prometheus. Routing is configured in `monitoring/alertmanager.yml`.

9.1 Alert Rule Table

Alert Name	Severity	For Duration	Condition & Description
`HighErrorRate`	critical	2 min	`rate(errors[5m]) / rate(requests[5m]) > 0.05` — Error rate exceeds 5%.
`DataDriftDetected`	warning	5 min	`model_drift_score > 0.3` — PSI exceeds 0.3, indicating input distribution shift.
`HighPredictionLatency`	warning	3 min	`histogram_quantile(0.99, ...) > 2` — p99 latency over 2 seconds.
`HighCPUUsage`	warning	5 min	CPU idle $< 20\%$, i.e. usage $> 80\%$.
`HighMemoryUsage`	critical	5 min	Available memory $< 15\%$, i.e. usage $> 85\%$.
`ServiceDown`	critical	1 min	`up == 0` — Any monitored target is unreachable.
`NoPredictionTraffic`	critical	5 min	`rate(requests[5m]) == 0` — No inference requests received.
`TrafficSpike`	warning	2 min	1-minute request rate $> 3\times$ the 5-minute baseline.
`HighErrorCount`	critical	2 min	`rate(prediction_errors_total[5m]) > 10` — More than 10 errors/second.
`HighLoadAverage`	warning	5 min	`node_load1 > 4` — System 1-minute load average too high.

9.2 Alertmanager Configuration

Figure 25 shows the Alertmanager UI confirming the routing configuration and current alert state.

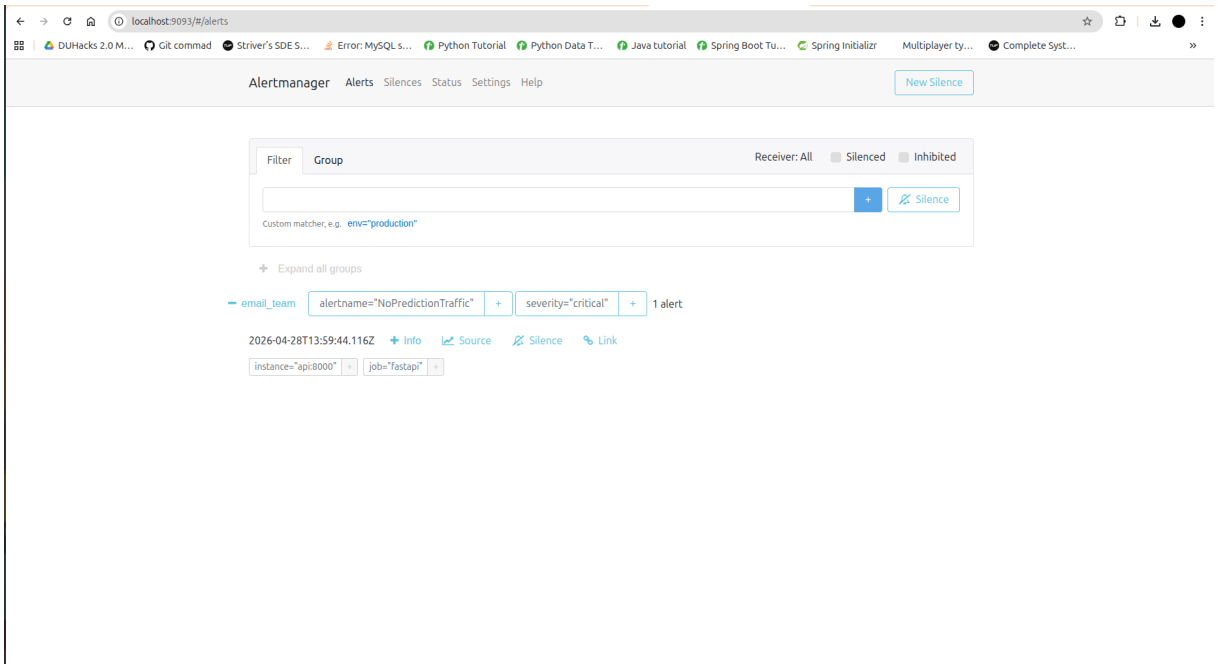


Figure 25: Alertmanager — Alert routing and status UI

10 References

- Dataset: Kaggle — `mayank0bhardwaj/alphabets`
- MLflow Documentation: `https://mlflow.org/docs/latest`
- DVC Documentation: `https://dvc.org/doc`
- Apache Airflow Documentation: `https://airflow.apache.org/docs`
- Prometheus Documentation: `https://prometheus.io/docs`
- Grafana Documentation: `https://grafana.com/docs`
- MediaPipe Hands: `https://developers.google.com/mediapipe/solutions/vision/hand_landmarker`